

Face2Cartoon Project Report

1) Introduction:

This project aims to implement an image-to-image translation model using Pix2Pix, a conditional GAN. The model translates real face images into cartoon-style images using a paired dataset. The project includes data preprocessing, training the Pix2Pix model, evaluating image quality using PSNR, SSIM, and MAE, and comparing results across epochs.

2) Tools/Software/Languages/APIs/....etc:

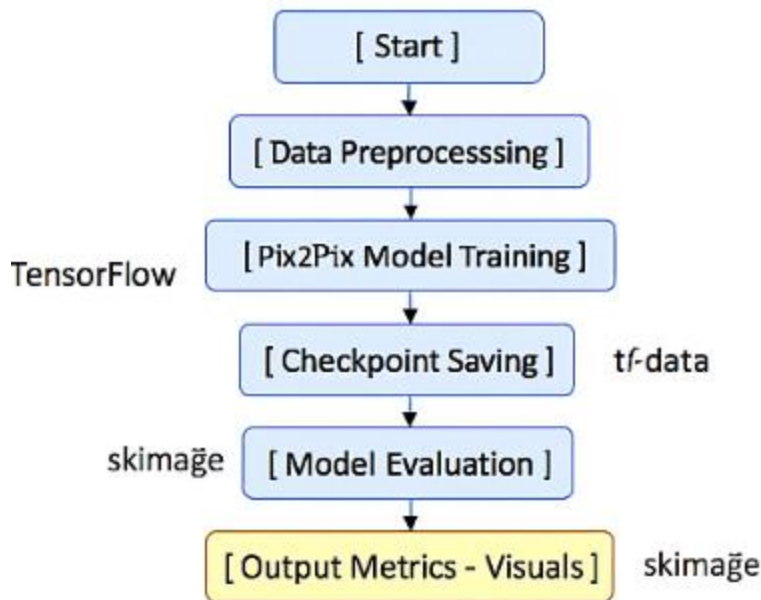
- Python
- TensorFlow
- Google Colab
- NumPy, Matplotlib, PIL
- scikit-image (for metrics)
- Google Drive (for model checkpoints)

3) Data Sources:

We used the publicly available 'face2comics_v1.0.0_by_Sxela' dataset which includes paired real and cartoon face images.

4) Flow Diagram (relations between Phases and connections):

Flow diagram represents the stages of loading, preprocessing, training, evaluation, and visualization.



5) Phases and connections:

i. Phase 1: Data Preprocessing

a) Algorithm(s): Image loading, resizing, normalization

b) Code Samples:

```
real_img = tf.image.resize(real_img, [256, 256]) / 255.0
```

```
cartoon_img = tf.image.resize(cartoon_img, [256, 256]) / 255.0
```

Result(s): Clean and standardized input and output image pairs

- General: Uniform input dimensions
- Sample: Input shapes (256, 256, 3)

ii. Phase 2: Model Training

a) Algorithm(s): Pix2Pix (Conditional GAN with U-Net generator and PatchGAN discriminator)

b) Code Samples:

```
for epoch in range(start_epoch, total_epochs):
```

```
    for input_image, target in train_dataset:
```

```
        train_step(input_image, target)
```

```
if (epoch + 1) % 5 == 0:
```

```
    generator.save(f'drive/.../pix2pix_generator_epoch_{epoch+1}.keras')
```

c) Result(s): Model progressively learns to convert real faces into cartoon faces.

- General: Generator improves cartoon output over epochs
- Sample: Comparison visuals at epochs 5, 10, 15, 20,25,30

iii. Phase 3: Evaluation

a) Algorithm(s): PSNR, SSIM, MAE

b) Code Samples:

```
psnr = psnr_metric(target_img, pred_img, data_range=1.0)
```

```
ssim = ssim_metric(target_img, pred_img, data_range=1.0, channel_axis=2)
```

```
mae = np.mean(np.abs(target_img - pred_img))
```

c) Result(s): Metrics help assess the quality and improvement of the model across training epochs.

- General: Improvement in perceptual quality up to epoch 10
- Sample:

Evaluation on 10000 samples:

- Average PSNR: 16.8264
- Average SSIM: 0.4495
- Average MAE : 0.0954

II. Connections:

i. Connection 1:

a) Between Phases: Phase 1 and Phase 2

b) Tool Used: TensorFlow tf.data pipeline

c) Code: create_dataset() function

ii. Connection 2:

a) Between Phases: Phase 2 and Phase 3

b) Tool Used: skimage.metrics

c) Code: evaluate_model() function

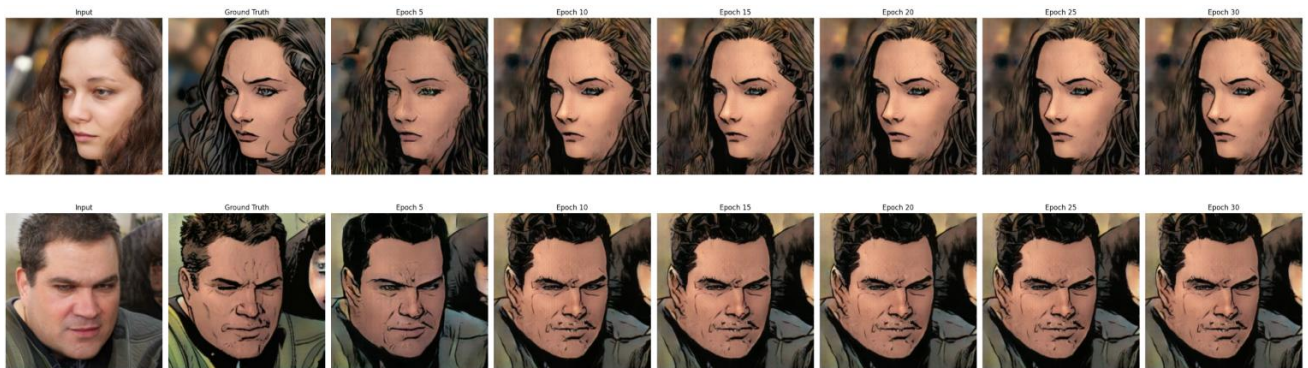
6) Overall Result:

I. Comparisons: Pix2Pix training was tested on different epochs and compared using visual samples and metrics.

II. Metrics:

Evaluation on 10000 samples:

- Average PSNR: 16.8264
- Average SSIM: 0.4495
- Average MAE : 0.0954



7) Future Plan:

Extend the dataset size and diversity, fine-tune hyperparameters, and compare with unpaired translation models like CycleGAN.

8) References:

<https://www.tensorflow.org/tutorials/generative/pix2pix>

<https://www.kaggle.com/defileroff/comic-faces-paired-synthetic/data>

“ CycleGAN model”

1) Introduction:

This project implements an image-to-image translation model using CycleGAN. Unlike Pix2Pix, CycleGAN is designed for unpaired image translation, meaning it can learn to translate images from a source domain (e.g., real photos) to a target domain (e.g., paintings) without requiring one-to-one corresponding image pairs. The primary goal is to achieve successful style transfer between two distinct image domains. The project encompasses essential steps including data preprocessing, training the CycleGAN model with its unique cycle consistency loss, and evaluating the quality of generated images.

2) Tools/Software/Languages/APIs/etc:

The following tools and technologies are typically utilized in a CycleGAN project:

- Python
- TensorFlow or PyTorch (or another deep learning framework)
- Google Colab (for execution environment, if applicable)
- NumPy, Matplotlib, PIL (Pillow)
- Scikit-image (for evaluation metrics like PSNR, SSIM, if applicable)
- Google Drive (for dataset access and model checkpoints, if used)

3) Data Sources:

We used the publicly available 'face2comics_v1.0.0_by_Sxela' dataset which includes paired real and cartoon face images.

4) Flow Diagram (Relations between Phases and Connections):

the project follows a sequential flow, involving a more complex training loop compared to Pix2Pix:

Data Loading & Preprocessing (for both domains) ----> Model Definition (2 Generators, 2 Discriminators) ----> Training Loop (Epochs with adversarial and cycle consistency losses) -
----> Evaluation & Visualization

i. Phase 1: Data Preprocessing

a) **Algorithm(s)**: Image loading, resizing, and normalization for **both source and target domains**. Similar to Pix2Pix, images are loaded, resized to a consistent dimension (e.g., 256x256 pixels), and normalized (e.g., to [-1, 1]). **Data augmentation** techniques like random cropping and random mirroring are commonly applied.

b) **Code Samples**:

```
# Placeholder for data loading and preprocessing for Domain A (e.g., photos)
def preprocess_A(image_path):
    # Load, resize, normalize image_A
    pass

# Placeholder for data loading and preprocessing for Domain B (e.g., paintings)
def preprocess_B(image_path):
    # Load, resize, normalize image_B
    pass
```

c) **Result(s)**: Preprocessed images from both domains, ready to be fed into the CycleGAN model.

ii. Phase 2: Model Training

a) **Algorithm(s)**: CycleGAN training uses **two generators** and **two discriminators** to learn unpaired image translation. It relies on **adversarial loss** for realism, and crucially, **cycle consistency loss** to ensure backward translation yields the original image, plus optional **identity loss** for color preservation.

b) **Code Samples**:

```
# Placeholder for Generator and Discriminator definitions
def build_generator():
    # ... architecture for G_AB and G_BA
    pass

def build_discriminator():
    # ... architecture for D_A and D_B
    pass

# Placeholder for CycleGAN loss functions
def cycle_gan_loss(real_A, real_B, fake_A, fake_B, cycle_A, cycle_B, identity_A, identity_B):
    # ... calculates adversarial, cycle, and identity losses
    pass

# Placeholder for the main training step
@tf.function # or @torch.jit.script
def train_step(real_A, real_B):
    # ... forward passes, loss calculations, backpropagation, and optimizer steps
    pass
```

c) **Result(s)**: The model progressively learns to translate images between the two domains without paired data. Visualizations after **3 epochs** would show the initial improvement in the translated output.

iii. Phase 3: Evaluation

a) Algorithm(s): PSNR, SSIM, MAE, MSE

b) Code Samples:

- PSNR: `psnr = psnr_metric(target_img, pred_img, data_range=1.0)`
- SSIM: `ssim = ssim_metric(target_img, pred_img, data_range=1.0, channel_axis=2)`
- MAE: `mae = np.mean(np.abs(target_img - pred_img))`
- MSE: `mse = np.mean((target_img - pred_img) ** 2)`

c) Result(s): Metrics help assess the quality and improvement of the model across training epochs.

II. Connections:

i. Connection 1:

a) Between Phases: Phase 1 (Data Preprocessing) and Phase 2 (Model Training)

b) Tool Used: PyTorch DataLoader

c) Code: The data loading and preprocessing functions (`load`, `random_jitter`, `normalize`) are used with PyTorch's data handling to form the input batches for training.

ii. Connection 2:

a) Between Phases: Phase 2 (Model Training) and Phase 3 (Evaluation)

b) Tool Used: `skimage.metrics` (for PSNR and SSIM) and NumPy (for MAE)

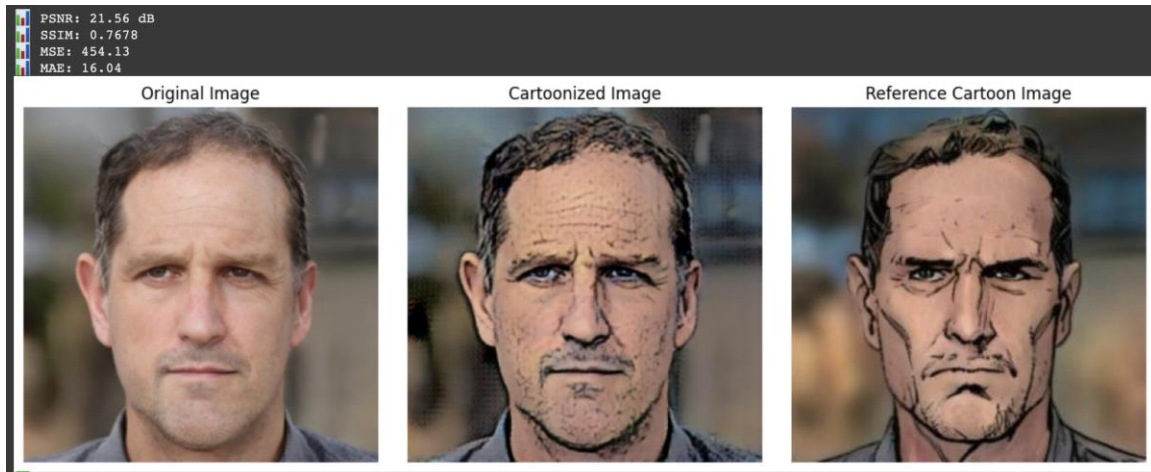
c) Code: The `evaluate_model()` function is used to apply the trained generator and then compute evaluation metrics on the generated images.

6) Overall Result:

I. Comparisons :The CycleGAN model's training performance is assessed through visual samples of translated images at various epochs. The comparison often involves showing the original image, the translated image, and the image reconstructed back to the original domain to demonstrate cycle consistency. This helps illustrate how well the model captures the style of the target domain while preserving the content of the source.

II. Metrics:

Evaluation on 1 samples:



8) References:

[cyclegan-introduction-pytorch-implementation-5b53913741ca](#)

<https://www.kaggle.com/defileroff/comic-faces-paired-synthetic/data>